第一周

Git 版本控制
零基础实战手册

从零开始，五天掌握 Git 核心技能

适合人群：零基础学员 | 物联网工程专业

学习目标：能够独立使用 Git 管理项目代码

目　录

目录

目　录 2
本周学习目标 4
第一章　概念引入——什么是版本控制？ 4
第二章　环境准备——手把手教你安装配置 6
第三章　五天冲刺计划——从零到实战 9
第四章　避坑指南——新手最常犯的错误 14
第五章　自测清单——检验你的学习成果 16
附录　Git 命令速查表 16

本周学习目标

通过本周的学习，你将建立起对版本控制系统的完整认知，并掌握 Git 的核心操作技能。具体来说，你需要达成以下四个层面的学习目标：

目标层次	具体内容
概念理解	理解版本控制的核心概念，掌握工作区、暂存区、版本库三个区域的关系与作用，建立对 Git 工作流程的系统认知
环境搭建	独立完成 Git 的安装与配置，包括用户信息设置、SSH 密钥配置等，能够排查常见的环境问题
核心操作	熟练使用 git init、git add、git commit、git status、git log 等核心命令，能够完成代码的版本管理基本流程
问题解决	能够识别并解决常见的 Git 操作错误，如提交信息写错、文件误删等情况，具备基本的故障排查能力

第一章　概念引入——什么是版本控制？

1.1　用游戏存档理解版本控制

想象你正在玩一款角色扮演游戏。当你完成了一个重要任务、打败了一个强力 BOSS、或者获得了一件稀有装备时，你会做什么？没错，你会创建一个存档点。这样，如果后续的游戏进程不顺利，或者你想尝试不同的选择路线，你都可以随时回到之前的存档点重新开始。

版本控制系统（Version Control System，简称 VCS）就像是程序员的超级存档系统。它能够记录项目中每一个文件的每一次修改，让你可以随时查看历史版本、比较不同版本之间的差异、或者将项目恢复到之前的某个状态。与游戏存档不同的是，Git 的存档更加智能和高效——它不会简单地复制整个项目，而是只记录发生变化的部分，大大节省了存储空间。

让我们用一个更具体的例子来说明。假设你正在写一篇重要的论文，在没有版本控制的情况下，你可能会创建多个文件来保存不同的版本：论文_v1.doc、论文_v2.doc、论文_最终版.doc、论文_最终版_修改.doc、论文_最终版_修改_再改.doc……这种做法不仅文件管理混乱，而且很难追踪每次修改的具体内容。而使用 Git 后，你只需要维护一个文件，Git 会帮你记录每次修改的内容、时间和修改者，你可以随时查看任意历史版本，也可以轻松比较不同版本之间的差异。

☐ 核心价值

版本控制的核心价值在于：它让后悔变得不再可怕。你可以大胆地尝试新想法，因为任何修改都可以追溯和撤销。

1.2　为什么程序员必须会 Git？

在当今的软件开发领域，Git 已经成为事实上的行业标准。根据 2023 年的开发者调查数据，超过 90%的专业开发者使用 Git 进行版本控制。掌握 Git 不再是一项加分技能，而是程序员的基本功。以下是每个程序员必须掌握 Git 的几个核心理由：

第一，团队协作的必备工具。在现代软件开发中，一个项目往往由多人协作完成。Git 提供了完善的分支管理和合并机制，让多人可以同时在不同的功能上工作，最后将各自的修改整合到一起。没有 Git，团队协作将变得异常困难——你需要手动协调谁在修改哪个文件，如何合并不同人的修改，这简直是一场噩梦。

第二，代码安全的保障。代码是程序员最宝贵的资产。使用 Git，你的代码历史被完整地保存在版本库中。即使你误删了重要代码，或者引入了严重的 bug，都可以通过 Git 回退到之前的稳定版本。这种安全网让开发者可以更加自信地进行代码修改和重构。

第三，开源社区的通行证。GitHub、GitLab 等基于 Git 的代码托管平台已经成为开源项目的聚集地。如果你想参与开源项目、学习优秀的代码、或者展示自己的技术能力，Git 是必不可少的工具。许多公司在招聘时也会查看候选人的 GitHub 主页，作为评估技术能力的重要参考。

第四，持续集成与部署的基础。现代软件开发流程中，代码提交后会自动触发测试、构建、部署等一系列流程。这些自动化流程都建立在 Git 版本控制的基础之上。掌握 Git，是理解现代 DevOps 工作流程的第一步。

1.3　三个核心区域：工作区、暂存区、版本库

理解 Git 的工作原理，关键在于理解它的三个核心区域。这三个区域构成了 Git 工作流程的基础，也是初学者最容易混淆的地方。让我们通过一个形象的比喻来理解它们：

区域名称	形象比喻	详细说明
工作区 (Working Directory)	你的办公桌	你实际编辑文件的地方，能看到、能修改的所有项目文件都在这里。这是你日常工作的区域，所有的代码修改都从这里开始。
暂存区 (Staging Area)	待打包的箱子	你准备提交的修改暂存的地方。你可以选择性地将某些修改放入暂存区，而不是一次性提交所有修改。这让你可以精心组织每一次提交的内容。
版本库 (Repository)	档案室	Git 存储所有版本历史的地方。每次提交（commit）都会在版本库中创建一个新的快照，永久保存项目的状态。所有历史记录都在这里安全存储。

理解这三个区域的关系，是掌握 Git 的关键。一个完整的 Git 工作流程是这样的：你在工作区修改文件，然后用 git add 命令将修改添加到暂存区，最后用 git commit 命令将暂存区的内容提交到版本库。这个流程确保了每次提交都是经过精心组织的，而不是随意的。

> ☐ **常见误区**
> 很多初学者会忽略暂存区的存在，直接想要提交修改。记住：Git 的设计要求你必须先 add 再 commit，这个两步流程是有意义的——它让你有机会审视和整理即将提交的内容。

第二章　环境准备——手把手教你安装配置

2.1　Windows 系统安装 Git

在 Windows 系统上安装 Git 非常简单，只需要按照以下步骤操作即可。整个安装过程大约需要 5-10 分钟，请确保你的电脑有稳定的网络连接。

步骤一：下载安装包

访问 Git 官方网站 https://git-scm.com/download/win，页面会自动识别你的系统版本（32 位或 64 位），点击下载链接开始下载。如果你的网络访问 GitHub 较慢，也可以使用国内镜像站点下载，例如淘宝镜像或华为云镜像。下载完成后，你会得到一个名为 Git-x.x.x-64-bit.exe 的安装文件（x.x.x 代表版本号）。

步骤二：运行安装程序

双击下载的安装文件，会弹出安装向导。对于初学者，建议大部分选项保持默认设置即可。以下是几个关键选项的说明：

安装选项	建议选择与说明
选择默认编辑器	建议选择 Visual Studio Code 或 Notepad++，这两个编辑器对中文支持较好，学习曲线平缓。如果你熟悉 Vim，也可以选择 Vim。
PATH 环境配置	选择 Git from the command line and also from 3rd-party software，这样你可以在命令提示符和 PowerShell 中使用 Git 命令。
SSH 可执行文件	选择 Use bundled OpenSSH，使用 Git 自带的 SSH 工具，避免与系统其他 SSH 工具冲突。
HTTPS 传输后端	选择 Use the OpenSSL library，这是最常用的配置，兼容性最好。

步骤三：验证安装

安装完成后，打开命令提示符（按 Win+R，输入 cmd，回车）或 PowerShell，输入以下命令验证安装是否成功：

```
git --version
```

如果安装成功，你会看到类似 git version 2.43.0.windows.1 的输出，显示当前安装的 Git 版本号。如果提示 git 不是内部或外部命令，请尝试重启电脑，或者检查 PATH 环境变量是否正确配置。

2.2 Mac 系统安装 Git

Mac 系统安装 Git 有多种方式，这里介绍最常用的两种方法。选择其中一种即可，不必重复安装。

方法一：安装 Xcode 命令行工具（推荐）

这是最简单的方法。打开终端（Terminal），输入以下命令：

```
xcode-select --install
```

系统会弹出一个对话框，询问是否安装命令行工具。点击安装，等待下载完成即可。这个过程可能需要较长时间，取决于你的网络速度。安装完成后，Git 会自动配置好，可以直接使用。

方法二：使用 Homebrew 安装

如果你已经安装了 Homebrew（Mac 上的包管理器），可以使用以下命令安装 Git：

```
brew install git
```

Homebrew 会自动下载并安装最新版本的 Git，同时处理所有依赖关系。安装完成后，同样使用 git --version 命令验证安装是否成功。

2.3 初次配置：告诉 Git 你是谁

Git 安装完成后，第一件事就是配置你的身份信息。这是因为 Git 的每次提交都会记录提交者的信息，这对于团队协作和代码追溯非常重要。请务必使用真实信息，特别是如果你计划使用 GitHub 等平台的话。

打开终端或命令提示符，依次输入以下两条命令：

```
git config --global user.name "你的姓名"
git config --global user.email "你的邮箱"
```

例如，如果你的名字是张三，邮箱是 zhangsan@example.com，则输入：

```
git config --global user.name "张三"
```

```
git config --global user.email "zhangsan@example.com"
```

这里的--global 参数表示这是全局配置，会应用到你电脑上所有的 Git 项目。配置完成后，可以使用以下命令查看配置是否正确：

```
git config --list
```

> ☐ **小贴士**
> 建议将邮箱设置为与 GitHub 账号相同的邮箱，这样你的提交记录可以正确关联到你的 GitHub 账户，在个人主页上显示你的贡献统计。

2.4 配置 SSH Key：告别密码输入

SSH Key 是一种安全的身份验证方式，配置后你可以在与 GitHub 等平台通信时无需每次输入密码。虽然这一步不是必须的，但强烈建议配置，它会大大提升你的工作效率。

步骤一：生成 SSH 密钥

在终端中输入以下命令（将邮箱替换为你的 GitHub 邮箱）：

```
ssh-keygen -t ed25519 -C "你的邮箱"
ssh-keygen -t 秘钥类型-C "你的邮箱"
```

系统会提示你选择保存位置和设置密码。对于初学者，建议直接按三次回车，使用默认设置（不设置密码）。生成的密钥对会保存在~/.ssh/目录下，其中 id_ed25519 是私钥（必须保密），id_ed25519.pub 是公钥（需要上传到 GitHub）。

步骤二：添加公钥到 GitHub

首先，查看并复制你的公钥内容：

```
cat ~/.ssh/id_ed25519.pub
# Windows 用户使用：type %USERPROFILE%\.ssh\id_ed25519.pub
```

复制输出的内容（以 ssh-ed25519 开头的一长串字符）。然后登录 GitHub，点击右上角头像 → Settings → SSH and GPG keys → New SSH key，将复制的内容粘贴到 Key 文本框中，Title 可以随意填写（如 My Laptop），最后点击 Add SSH key。

步骤三：测试连接

输入以下命令测试 SSH 连接是否成功：

```
ssh -T git@github.com
```

如果看到 Hi username! You have successfully authenticated...的提示，说明配置成功。如果提示连接失败，请检查密钥是否正确添加到 GitHub，以及网络是否正常。

第三章　五天冲刺计划——从零到实战

本章将带你通过五天的系统学习，从零开始掌握 Git 的核心技能。每天的学习内容都经过精心设计，既有理论讲解，也有动手实践。建议每天投入 1-2 小时，按顺序完成学习任务。记住：学习编程最好的方式就是动手实践，不要只是看，要跟着做！

Day 1：创建你的第一个仓库

今天的学习目标是理解 Git 仓库的概念，并亲手创建你的第一个仓库。这是 Git 学习的第一步，也是最基础的一步。

核心概念：什么是仓库？

仓库（Repository）是 Git 管理项目的基本单位。你可以把它理解为一个被 Git 监控的文件夹——Git 会记录这个文件夹内所有文件的变化历史。一个仓库可以是本地的（只存在于你的电脑上），也可以是远程的（存储在 GitHub 等平台上）。

有两种创建 Git 仓库的方式：第一种是在现有文件夹中初始化仓库，第二种是从远程平台克隆一个已存在的仓库。今天我们重点学习第一种方式。

动手实践：创建并初始化仓库

首先，创建一个新文件夹作为你的项目目录：

```
# 创建项目文件夹
mkdir my-first-repo
# 进入项目文件夹
cd my-first-repo
```

然后，在这个文件夹中初始化 Git 仓库：

```
git init
```

执行后，你会看到 Initialized empty Git repository 的提示。此时，Git 会在当前文件夹中创建一个隐藏的.git 目录，用于存储版本历史。你可以使用 ls -a（Mac/Linux）或 dir /a（Windows）命令查看这个隐藏目录。

☐ 理解.git 目录

.git 目录是 Git 仓库的核心，它存储了所有的版本历史、配置信息等。删除这个目录，就相当于删除了整个仓库的历史记录，项目会变回普通文件夹。所以，千万不要手动修改或删除.git 目录中的内容！

Day 2：掌握核心三剑客——add、commit、status

今天你将学习 Git 最核心的三个命令，这三个命令构成了 Git 工作流程的基础。掌握它们，你就能够完成基本的版本控制操作。

git status：查看当前状态

git status 是 Git 中最常用的命令之一，它告诉你当前仓库的状态：哪些文件被修改了、哪些文件在暂存区、当前在哪个分支等。养成随时使用 git status 查看状态的习惯，这能帮助你更好地理解 Git 的工作流程。

在 my-first-repo 目录中创建一个新文件，然后查看状态：

```
# 创建一个新文件
echo "Hello Git" > readme.txt
# 查看状态
git status
```

你会看到 readme.txt 显示为 Untracked files（未跟踪文件），这意味着 Git 发现了这个新文件，但还没有开始跟踪它的变化。

git add：添加到暂存区

git add 命令用于将文件的修改添加到暂存区。暂存区是 Git 的一个重要概念，它让你可以选择性地提交修改，而不是一次性提交所有修改。

将 readme.txt 添加到暂存区：

```
git add readme.txt
# 再次查看状态
git status
```

现在你会看到 readme.txt 显示为 Changes to be committed（待提交的修改），文件名显示为绿色（在支持颜色的终端中）。这表示文件已经进入暂存区，准备好被提交了。

git add 还有几个常用的变体：

```
git add .           # 添加当前目录下所有修改
git add -A          # 添加所有修改（包括删除）
git add *.txt       # 添加所有 txt 文件
```

git commit：提交到版本库

git commit 命令将暂存区的内容永久保存到版本库中，创建一个新的版本快照。每次提交都需要附带一条提交信息（commit message），描述这次提交的内容。

提交你的第一次修改：

```
git commit -m "添加 readme 文件"
```

执行后，Git 会显示提交的信息，包括提交 ID（一串唯一的哈希值）、提交者信息、提交日期等。恭喜你，你已经完成了第一次 Git 提交！

☐ 提交信息规范

好的提交信息应该简洁明了地描述本次提交的内容。建议使用中文，遵循动词+对象的格式，如修复登录 bug、添加用户列表页面、优化数据库查询等。避免使用更新、修改这类模糊的描述。

Day 3：追溯历史——log 与 diff 命令

今天你将学习如何查看项目的历史记录和修改差异。这些技能对于理解项目演变过程、定位问题来源非常重要。

git log：查看提交历史

git log 命令显示项目的提交历史，包括每次提交的 ID、作者、日期和提交信息。这是了解项目演变过程的重要工具。

首先，让我们多创建几次提交，以便有历史可看：

```
# 修改文件
echo "这是第二行" >> readme.txt
git add readme.txt
git commit -m "添加第二行内容"

# 再次修改
echo "这是第三行" >> readme.txt
git add readme.txt
git commit -m "添加第三行内容"
```

现在查看提交历史：

```
git log
```

你会看到按时间倒序排列的所有提交记录。每条记录包含提交 ID（commit hash）、作者信息、提交日期和提交信息。

git log 有几个常用的选项，可以让输出更易读：

```
git log --oneline           # 简洁模式，每条提交只显示一行
git log --graph             # 显示分支图形
git log -n 3                # 只显示最近 3 条提交
git log --oneline --graph --all  # 组合使用，推荐！
```

git diff：查看修改差异

git diff 命令用于比较不同版本之间的差异，或者查看未提交的修改。这是代码审查和问题排查的重要工具。

修改 readme.txt 但不提交，然后查看差异：

```
# 修改文件
echo "这是新添加的一行" >> readme.txt
# 查看工作区与暂存区的差异
git diff
```

输出会显示具体的修改内容：以-开头的行表示删除的内容，以+开头的行表示新增的内容。这种格式称为统一差异格式（unified diff format），是程序员交流代码修改的标准方式。

其他常用的 diff 命令：

```
git diff --staged           # 查看暂存区与最新提交的差异
git diff HEAD               # 查看工作区与最新提交的差异
git diff commit1 commit2    # 比较两个提交之间的差异
```

Day 4：时光穿梭——checkout 与 reset

今天你将学习 Git 最强大的功能之一：版本回退。Git 允许你随时回到项目的任意历史版本，这是版本控制系统的核心价值所在。

git checkout：切换版本与分支

git checkout 命令有多种用途，今天我们学习如何用它来查看历史版本。首先，使用 git log --oneline 查看提交历史，找到你想回到的版本 ID（前几位即可）：

```
# 查看历史
git log --oneline
# 假设第一个提交的 ID 是 a1b2c3d，切换到那个版本
git checkout a1b2c3d
```

此时，你的项目文件会恢复到那个版本的状态。注意，你现在处于游离 HEAD 状态（detached HEAD），可以查看历史代码，但不建议在这种状态下进行开发。

要回到最新版本，使用：

```
git checkout main
# 或者
git checkout master
# 具体取决于你的主分支名称
```

git reset：版本回退

git reset 是真正回退版本的命令，它会移动 HEAD 指针，改变当前的提交历史。有三种模式：

模式	说明与使用场景
--soft	只移动 HEAD 指针，保留工作区和暂存区。适用于：想撤销提交但保留修改，稍后重新组织提交。
--mixed	移动 HEAD 指针，重置暂存区，保留工作区。这是默认模式。适用于：想撤销 add 操作，重新选择要提交的内容。
--hard	移动 HEAD 指针，重置暂存区和工作区。完全丢弃所有修改。慎用！适用于：确定要放弃所有修改，回到干净状态。

☐ 危险操作警告

git reset --hard 会永久丢失未提交的修改，无法恢复！在使用这个命令之前，请确保你真的想丢弃所有修改。如果不确定，可以先使用 git stash 保存当前修改。

Day 5：远程仓库——连接 GitHub

今天你将学习如何将本地仓库与 GitHub 远程仓库关联，实现代码的云端备份和团队协作。这是 Git 工作流程中非常重要的一环。

创建 GitHub 仓库

首先，登录 GitHub，点击右上角的+按钮，选择 New repository。填写仓库名称（如 my-first-repo），选择 Public 或 Private，不要勾选 Add a README file（因为我们本地已经有内容了）。点击 Create repository 创建仓库。

关联远程仓库

在本地仓库中，添加远程仓库地址：

```
# 添加远程仓库（将下面的用户名替换为你的 GitHub 用户名）
git remote add origin git@github.com:用户名/my-first-repo.git
# 验证远程仓库
git remote -v
```

推送代码到 GitHub

将本地代码推送到 GitHub：

```
# 推送到远程仓库
git push -u origin main
# 如果你的主分支叫 master，使用：
git push -u origin master
```

-u 参数表示设置上游分支，设置后以后只需要 git push 即可推送。刷新 GitHub 页面，你会看到你的代码已经出现在仓库中了！

克隆远程仓库

如果你想获取别人在 GitHub 上的项目，可以使用 git clone 命令：

```
git clone git@github.com:用户名/仓库名.git
```

这会在当前目录创建一个与仓库同名的文件夹，包含项目的所有文件和完整的 Git 历史。

第四章　避坑指南——新手最常犯的错误

学习 Git 的过程中，有些错误几乎是每个初学者都会犯的。本章总结了最常见的错误及其解决方法，帮助你少走弯路。

错误一：提交信息写错了

场景：你刚刚执行了 git commit -m 修复 bug，但发现提交信息应该更具体，比如修复用户登录时的空指针异常。

解决方法：使用--amend 参数修改最近一次提交的信息：

```
git commit --amend -m "修复用户登录时的空指针异常"
```

> ☐ **注意**
> 如果你已经将提交推送到远程仓库，修改提交信息后需要使用 git push --force 强制推送。但在团队

协作中要慎用强制推送，因为它可能覆盖别人的提交。

错误二：漏掉了某个文件

场景：你刚刚提交了代码，但发现漏掉了一个文件没有添加。

解决方法：添加漏掉的文件，然后使用--amend 将其加入最近一次提交：

```
git add 漏掉的文件
git commit --amend --no-edit
```

--no-edit 参数表示保留原来的提交信息不变。

错误三：误删了重要文件

场景：你使用 rm 命令删除了一个文件，但后来发现还需要它。

解决方法：如果文件已经被 Git 跟踪，可以使用 checkout 命令恢复：

```
git checkout -- 文件名
# 或者使用更新的 restore 命令
git restore 文件名
```

如果文件已经被提交过，即使你用 git rm 删除了，也可以从历史中恢复：

```
# 从指定提交中恢复文件
git checkout 提交 ID -- 文件名
```

错误四：想撤销 git add

场景：你使用 git add 添加了文件到暂存区，但发现添加错了，想撤销。

解决方法：使用 restore 命令将文件从暂存区移除：

```
git restore --staged 文件名
# 或者使用传统的 reset 命令
git reset HEAD 文件名
```

这只会将文件从暂存区移除，不会影响工作区的文件内容。

错误五：想撤销所有未提交的修改

场景：你修改了很多文件，但想把它们全部恢复到上次提交的状态。

解决方法：

```
# 恢复所有工作区的修改
git restore .
# 或者
git checkout -- .
```

☐ 再次提醒

这个操作不可逆！所有未提交的修改都会丢失。使用前请确认你真的想放弃这些修改。

第五章　自测清单——检验你的学习成果

完成本周学习后，请对照以下清单检验自己的掌握程度。对于每项技能，尝试不看文档独立完成操作。

基础概念自测

检测项目	掌握程度
能够用自己的话解释版本控制的概念	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够区分工作区、暂存区、版本库三个区域	☐ 未掌握　☐ 基本掌握　☐ 熟练
理解 Git 工作流程：修改→add→commit	☐ 未掌握　☐ 基本掌握　☐ 熟练
知道什么是仓库，理解本地仓库与远程仓库的关系	☐ 未掌握　☐ 基本掌握　☐ 熟练

实操技能自测

检测项目	掌握程度
能够独立创建并初始化 Git 仓库	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够使用 git add 和 git commit 提交修改	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够使用 git status 查看仓库状态	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够使用 git log 查看提交历史	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够使用 git diff 查看修改差异	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够使用 git reset 回退版本	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够将本地仓库推送到 GitHub	☐ 未掌握　☐ 基本掌握　☐ 熟练
能够克隆远程仓库到本地	☐ 未掌握　☐ 基本掌握　☐ 熟练

附录　Git 命令速查表

以下是本周学习的 Git 命令汇总，建议打印出来放在手边随时查阅。

配置命令

命令	说明
`git config --global user.name "姓名"`	设置全局用户名
`git config --global user.email "邮箱"`	设置全局邮箱
`git config --list`	查看所有配置

仓库操作

命令	说明
`git init`	在当前目录初始化仓库
`git clone <url>`	克隆远程仓库
`git status`	查看仓库状态

提交操作

命令	说明
`git add <file>`	添加文件到暂存区
`git add .`	添加所有修改到暂存区
`git commit -m "信息"`	提交暂存区到版本库
`git commit --amend`	修改最近一次提交

历史与差异

命令	说明
`git log`	查看提交历史
`git log --oneline`	简洁模式查看历史
`git diff`	查看工作区与暂存区差异
`git diff --staged`	查看暂存区与最新提交差异

撤销与回退

命令	说明
`git restore <file>`	丢弃工作区的修改
`git restore --staged <file>`	将文件从暂存区移除
`git reset HEAD~1`	撤销最近一次提交（保留修改）
`git reset --hard HEAD~1`	撤销最近一次提交（丢弃修改）

远程操作

命令	说明
`git remote add origin <url>`	添加远程仓库
`git push -u origin main`	推送到远程仓库并设置上游分支
`git push`	推送到远程仓库
`git pull`	从远程仓库拉取更新
`git remote -v`	查看远程仓库信息

恭喜你完成了 Git 版本控制的第一周学习！通过本周的学习，你已经掌握了 Git 的核心概念和基本操作，能够独立使用 Git 管理项目代码。但这只是 Git 强大功能的冰山一角，还有分支管理、合并冲突、变基（rebase）、标签管理等进阶内容等待你去探索。

记住：Git 是工具，不是目的。我们学习 Git 是为了更好地管理代码、更高效地协作。从下周开始，你将用 Git 来管理你的物联网项目代码，在实践中巩固所学。

最后，送你一句话：代码可以重写，但 Git 历史会记住你走过的每一步。祝你学习顺利，早日成为 Git 高手！